



UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA
BACHARELADO EM INTELIGÊNCIA ARTIFICIAL
INTERNET DAS COISAS
IWENS GERVASIO SENE JUNIOR

Grupo 2

Carlos Rocha
Pedro Coelho
Rafael Batista
Herik Kauan
Gabriel Oliveira

Atividade Prática 02 — Comunicação em IoT utilizando MQTT

Projeto: Sistema IoT para Segurança de Usinas Solares

Etapa 1 — Revisão da Atividade 01

Na Atividade 01 desenvolvemos uma solução para um problema real do setor de energia solar: usinas fotovoltaicas ocupam grandes áreas rurais e afastadas, com vigilância limitada, o que as torna alvo de furto de painéis, cabeamento de cobre e equipamentos dos inversores. A solução proposta é um sistema de monitoramento e segurança baseado em IoT, com sensores de perímetro (PIR), sensores magnéticos em portões, sensores de vibração/inclinação em racks de painéis, câmeras IP e atuadores de dissuasão (sirene e iluminação).

A arquitetura definida na Atividade 01 já previa três camadas: campo (sensores/atuadores + gateway LoRaWAN), nuvem (AWS IoT Core, banco de dados, backend de regras) e usuários finais (dashboard web e aplicativo mobile), mas sem detalhar o protocolo de comunicação entre essas camadas. Agora, na Atividade 02, iremos preencher essa lacuna, utilizando o protocolo MQTT como meio de comunicação entre os dispositivos.

Etapa 2 — Publisher, Broker e Subscriber

A arquitetura foi revisada para deixar explícitos os três papéis do MQTT. Os nós de campo (ESP32) atuam como Publishers, publicando eventos de sensores em tópicos específicos. Um Broker MQTT centraliza o roteamento das mensagens (HiveMQ). Do lado dos Subscribers estão o backend/motor de regras (que assina toda a telemetria e decide as respostas), o banco de dados, o dashboard web, o

aplicativo mobile e os próprios atuadores, que assinam tópicos de comando para saber quando ligar a sirene ou a iluminação.

Um ponto importante da arquitetura é que a comunicação é bidirecional: os sensores publicam telemetria (campo -> broker -> backend), enquanto o backend publica comandos (backend -> broker -> atuadores) após aplicar a lógica de correlação de eventos. Isso caracteriza os atuadores como Subscribers, e o backend como Publisher e Subscriber ao mesmo tempo — um padrão comum em arquiteturas MQTT de controle e monitoramento.

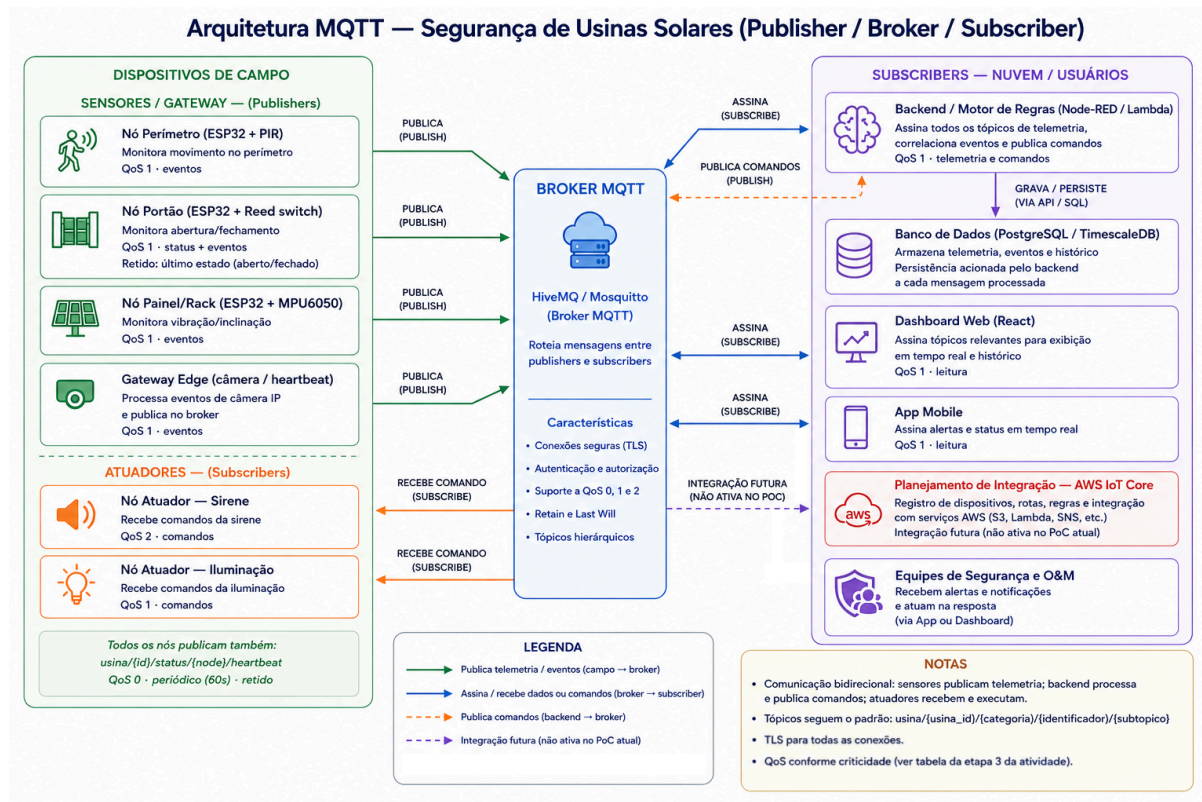


Figura 1 — Arquitetura MQTT: Publisher (campo) → Broker → Subscriber (nuvem, dashboard e atuadores).

Observação sobre o hardware: para fins desta simulação, cada nó de campo é representado como um cliente MQTT conectado diretamente via Wi-Fi ao broker. Na implementação final, os nós se comunicarão entre si e com o gateway via LoRaWAN (devido às grandes distâncias em campo), e será o gateway/computador de borda (Raspberry Pi) quem efetivamente publicará no broker MQTT em nome de todos os nós — a lógica de tópicos e mensagens definida aqui permanece a mesma.

Etapa 3 — Tópicos MQTT, Formato das Mensagens, QoS e Frequência

Todos os tópicos seguem o padrão hierárquico usina/{usina_id}/{categoria}/{identificador}/{subtopico}, o que facilita filtros por usina, por tipo de sensor e por nó específico usando wildcards (+ e #).

Tópico	Publisher → Subscriber	QoS	Retido	Frequência
usina/{id}/perimetro/{node}/evento	Nó perímetro → Backend	1	Não	Orientado a evento (movimento detectado)
usina/{id}/portao/{node}/status	Nó portão → Backend	1	Sim	Evento + heartbeat a cada 5 min
usina/{id}/paineis/{setor}/vibracao	Nó painel/rack → Backend	1	Não	Orientado a evento (limiar de inclinação)
usina/{id}/camera/{cam}/evento	Gateway edge → Backend	1	Não	Orientado a evento (disparo por sensor associado)
usina/{id}/status/{node}/heartbeat	Todos os nós → Backend	0	Sim	Periódico, a cada 60 s
usina/{id}/alerta	Backend → Dashboard/App	1	Não	Orientado a evento (pós correlação)
usina/{id}/atuador/{node}/sirene/cmd	Backend → Nó sirene	2	Não	Orientado a evento (comando de ação)
usina/{id}/atuador/{node}/luz/cmd	Backend → Nó iluminação	1	Não	Orientado a evento (comando de ação)

Justificativa das escolhas de QoS

- QoS 0 (heartbeat): perda ocasional é aceitável, pois o próximo heartbeat chega em poucos segundos; prioriza-se o baixo overhead.
- QoS 1 (maioria dos tópicos de evento e comando de iluminação): garante que a mensagem chegue ao menos uma vez — essencial para não perder um alerta de segurança, mesmo que ocasionalmente seja processada em duplicidade.
- QoS 2 (comando da sirene): garante entrega exatamente uma vez, evitando o risco de o atuador repetir o acionamento indevidamente.
- Retido (retain): usado no status do portão e nos heartbeats, para que um novo assinante (ex.: dashboard recém-conectado) receba imediatamente o último estado conhecido, sem esperar o próximo evento.

Formato das mensagens (payload em JSON)

Todas as mensagens usam JSON, com carimbo de tempo em ISO 8601 (UTC). Exemplo do tópico de evento de perímetro:

```
{
  "ts": "2026-07-27T18:10:03+00:00",
  "tipo": "movimento",
  "valor": 1,
  "bateria_pct": 87.4
}
```

Exemplo do tópico de comando para a sirene:

```
{
  "ts": "2026-07-27T18:10:03+00:00",
  "acao": "ligar",
  "duracao_seg": 30
}
```

Etapa 4 — Simulação com Broker MQTT Público

A prova de conceito foi implementada em Python, usando a biblioteca paho-mqtt e o broker público broker.hivemq.com (porta 1883). Dois scripts compõem a simulação: publisher.py, que representa os nós de campo, e subscriber.py, que representa o backend/motor de regras — o mesmo papel que futuramente será desempenhado pelo Node-RED ou por uma função AWS Lambda.

publisher.py — simula os nós de campo

Cada nó é modelado como um cliente MQTT independente, com Last Will and Testament (LWT) configurado: caso o dispositivo perca a conexão de forma anormal, o próprio broker publica automaticamente um status "offline" em seu tópico de heartbeat, permitindo que o backend detecte falhas de rede em campo.

```
class NoDeCampo:
    def __init__(self, node_id, topico_evento, qos, gerador_payload):
        self.client = mqtt.Client(client_id=f"pub-{node_id}")
        # Last Will: avisa o backend se o nó cair sem se desconectar
        self.client.will_set(
            topico_heartbeat,
            payload=json.dumps({"ts": agora_iso(), "estado": "offline"}),
            qos=1, retain=True,
        )

    def publicar_evento(self):
        payload = self.gerador_payload()
        self.client.publish(self.topico_evento, json.dumps(payload),
                           qos=self.qos, retain=self.retido)
```

subscriber.py — simula o backend / motor de regras

O subscriber se conecta ao broker e assina o wildcard usina/{id}/# — ou seja, toda a telemetria da usina em um único tópico de assinatura. Ao identificar um evento relevante (movimento no perímetro, ou vibração de painel acima de um limiar), ele aplica uma regra simples de correlação e publica de volta dois comandos de atuador e um alerta consolidado, fechando o ciclo publish → broker → subscribe → publish:

```
def on_connect(client, userdata, flags, reason_code, properties=None):
    client.subscribe(f"{PREFIXO}/usina/{USINA_ID}/#", qos=1)

def on_message(client, userdata, msg):
    payload = json.loads(msg.payload.decode())
    print(f"[RECEBIDO] {msg.topic} -> {payload}")
```

```

if "/perimetro/" in msg.topic and msg.topic.endswith("/evento"):
    _responder_incidente(client, origem="perimetro",
                           trecho=msg.topic.split('/')[-2])

def _responder_incidente(client, origem, trecho):
    client.publish(topico("atuador", trecho, "sirene", "cmd"),
                   json.dumps({"acao": "ligar", "duracao_seg": 30}), qos=2)
    client.publish(topico("alerta"),
                   json.dumps({"nivel": "alto", "origem": origem,
                              "trecho": trecho}), qos=1)

```

Execução e saída esperada

Para executar a simulação (localmente ou em um ambiente com acesso à internet, como Codespaces do GitHub):

```

pip install -r requirements.txt
python3 subscriber.py      # terminal 1
python3 publisher.py       # terminal 2

```

Saída esperada no terminal do subscriber, mostrando o ciclo completo de recebimento do evento e publicação da resposta (sirene, iluminação e alerta):

```

$ python3 subscriber.py
Conectado ao broker com codigo: Success
Inscrito em: usina/us01/#

[RECEBIDO] usina/us01/perimetro/perimetro-01/evento
-> {'ts': '2026-07-27T18:10:03+00:00', 'tipo': 'movimento', 'valor': 1,
'bateria_pct': 87.4}
[COMANDO] .../atuador/perimetro-01/sirene/cmd -> {'acao': 'ligar', 'duracao_seg': 30}
[COMANDO] .../atuador/perimetro-01/luz/cmd -> {'acao': 'ligar', 'duracao_seg': 120}
[ALERTA] .../alerta -> {'nivel': 'alto', 'origem': 'perimetro', 'trecho':
'perimetro-01'}

```

Etapa 5 — Organização do Projeto no GitHub

O projeto foi organizado em um repositório GitHub com a seguinte estrutura, contendo o código da simulação, os diagramas e a documentação:

```

.
├── README.md
├── docs/
│   └── Atividade02_Comunicacao_MQTT_Usinas_Solares.pdf
├── diagramas/
│   └── arquitetura_mqtt.png
└── simulacao/
    ├── publisher.py
    ├── subscriber.py
    └── requirements.txt

```

O README.md do repositório descreve o objetivo do projeto, a arquitetura MQTT resumida, a tabela de tópicos, as instruções de execução da simulação e o planejamento de integração com a AWS IoT Core.

Link do repositório: <https://github.com/raffaelbs/IoT-Atividade-02>

Etapa 6 — Planejamento de Integração com AWS IoT Core

O planejamento de migração para a AWS IoT Core prevê os seguintes elementos:

Device Registry e certificados: cada dispositivo (ESP32/gateway) será cadastrado individualmente no AWS IoT Core, com um certificado X.509 único, eliminando o uso de credenciais compartilhadas.

IoT Policies: políticas de acesso restringindo cada dispositivo a publicar e assinar apenas os tópicos que lhe cabem (ex.: um nó sensor só pode publicar em seu próprio tópico de evento, nunca em tópicos de comando de outros nós) — aplicando o princípio do menor privilégio.

Device Shadow: manutenção do último estado conhecido de cada dispositivo (ex.: estado do portão, nível de bateria), disponível mesmo quando o dispositivo está temporariamente offline.

IoT Rules Engine: regras SQL-like que roteiam mensagens recebidas para os serviços adequados — por exemplo, gravar telemetria no Amazon Timestream (série temporal), atualizar o estado atual no DynamoDB, armazenar mídia de eventos no Amazon S3, e disparar uma função AWS Lambda equivalente ao motor de correlação hoje simulado em Python/Node-RED.

Notificação e visualização: uso do Amazon SNS para disparo de notificações (push/SMS) às equipes de segurança, e do Amazon Location Service para exibir a localização dos eventos em um mapa da usina no dashboard.

Monitoramento da frota de dispositivos: uso do AWS IoT Device Management para acompanhar a saúde de todos os nós instalados (substituindo o heartbeat simplificado da simulação por métricas nativas da plataforma).

Essa transição preserva a estrutura de tópicos e o formato de mensagens já definidos nesta atividade — a principal mudança está na camada de transporte (TLS obrigatório, autenticação por certificado) e no destino das mensagens processadas, que passam a alimentar diretamente os serviços gerenciados da AWS.

Conclusão

Esta atividade evoluiu o projeto de segurança para usinas solares definido na Atividade 01, detalhando como os dispositivos efetivamente se comunicam por meio do protocolo MQTT. A definição clara de tópicos, formatos de mensagem, QoS e frequência de envio, somada a uma prova de conceito funcional com um broker público, cria uma base sólida e diretamente reaproveitável para a implementação do hardware definitivo e para a integração com a AWS IoT Core nas próximas etapas da disciplina.